

Construction Data Fetcher

คู่มืออธิบายโค้ด (src) แบบเข้าใจง่าย

สคริปต์ Python ที่ดึง ราคาวัสดุก่อสร้างจริง
จาก API ของหน่วยงานภาครัฐไทย
แล้วบันทึกลงไฟล์ Excel อัตโนมัติ

ไฟล์หลัก: fetch_construction_data.py

ภาษา: Python 3

แหล่งข้อมูล: กระทรวงพาณิชย์ (MOC) + ฉลากเขียว

ผลลัพธ์: ไฟล์ Excel (.xlsx) หลาย sheet

สารบัญ

1. ภาพรวม: สคริปต์นี้ทำอะไร	2
2. แหล่งข้อมูล: เราไปเอาข้อมูลจากไหน	3
3. โครงสร้างโค้ด: แต่ละส่วนทำหน้าที่อะไร	4
4. เจาะส่วนที่ 1: Config — กล่องตั้งค่า	5
5. เจาะส่วนที่ 2: http_get_json — คนวิ่งของ	6
6. เจาะส่วนที่ 3: การดึงแต่ละแหล่ง	7
7. เจาะส่วนที่ 4: write_excel — เขียนไฟล์ + จัดสวย	8
8. Data Pipeline: เส้นทางข้อมูลตั้งแต่ต้นจนจบ	9
9. ตัวอย่างจริง: Input / Process / Output	10
10. วิธีใช้งานจริง	12

| 1. ภาพรวม: สคริปต์นี้ทำอะไร

ลองนึกภาพแบบนี้ครับ — เราอยากรู้ว่า “ราคาวัสดุก่อสร้างในประเทศไทยตอนนี้เป็นอย่างไ” ปกติเราต้องเข้าเว็บราชการหลายเว็บ คลิกหาเอกสาร PDF โหลดมาเปิดอ่านทีละไฟล์ ซึ่งเสียเวลามาก และข้อมูลก็กระจัดกระจาย

สคริปต์ตัวนี้ทำงานแทนเราทั้งหมด มันจะ:

1. โทรไปถาม API ของหน่วยงานรัฐ 3 แห่ง (เหมือนโทรไปถามราคาทางโทรศัพท์ แต่เป็นคอมพิวเตอร์คุยกัน)
2. รับข้อมูลกลับมาในรูปแบบ JSON (รูปแบบข้อมูลที่เครื่องอ่านง่าย)
3. จัดระเบียบข้อมูลให้เป็นตาราง
4. เซฟลงไฟล์ Excel ที่เราเปิดดูได้เลย พร้อมแยกเป็นหลาย sheet

จุดเด่นสำคัญ: ข้อมูลทั้งหมดเป็น **ของจริง** ดึงสด ๆ จาก API ภาครัฐ ไม่ใช่ข้อมูลปลอม (mock) ที่พิมพ์ขึ้นมาเอง — รันเมื่อไหร่ก็ได้ข้อมูลล่าสุดเมื่อนั้น

ทำไมต้องเขียนสคริปต์ ไม่ทำมือ?

เพราะราคาวัสดุ **เปลี่ยนทุกเดือน** ถ้าทำมือ เราต้องมานั่งโหลดใหม่ทุกครั้ง แต่ถ้ามีสคริปต์ แค่รันคำสั่งเดียว `python3 fetch_construction_data.py` ก็ได้ไฟล์ Excel ใหม่พร้อมข้อมูลล่าสุดทันที ภายในไม่กี่นาที

| 2. แหล่งข้อมูล: เราไปเอาข้อมูลจากไหน

สคริปต์ดึงข้อมูลจาก API จริง 3 แหล่ง แต่ละแหล่งให้ข้อมูลคนละมุม:

แหล่งที่ 1 — CSI: ดัชนีราคาวัสดุก่อสร้างทั่วประเทศ

เจ้าของ: กระทรวงพาณิชย์ (สำนักงานนโยบายและยุทธศาสตร์การค้า)

URL: dataapi.moc.go.th/csi-indexes

ให้อะไร: ตัวเลขดัชนีราคาวัสดุก่อสร้างทั่วประเทศ รายเดือน ว่าเดือนนี้แพงขึ้นหรือถูกลงกี่เปอร์เซ็นต์เทียบกับปีก่อน

เปรียบเทียบ: เหมือน “เทอร์โมมิเตอร์วัดราคา” ของทั้งวงการก่อสร้าง

แหล่งที่ 2 — CPIP: ดัชนีราคารายจังหวัด

เจ้าของ: กระทรวงพาณิชย์

URL: dataapi.moc.go.th/cpip-indexes

ให้อะไร: ดัชนีราคาก่อสร้างแยก รายจังหวัด (มากถึง 77 จังหวัด) รายเดือน ใช้สำหรับงานก่อสร้างภาครัฐ

เปรียบเทียบ: เหมือนแผนที่ราคา บอกว่าจังหวัดไหนของแพง จังหวัดไหนของถูก

แหล่งที่ 3 — Green Label: ราคาวัสดุจริงเป็นบาท

เจ้าของ: ศูนย์ข้อมูลที่อยู่อาศัยแห่งชาติ (ฉลากเขียว)

URL: nhicservices.nha.co.th/.../green_label/mid

ให้อะไร: ราคาวัสดุ จริงเป็นบาท กว่า 5,500 รายการ มียี่ห้อ สเปค หน่วยครบ

เปรียบเทียบ: เหมือนใบเสนอราคาจริง ที่บอกว่า “ปูนยี่ห้อนี้ถูกละเท่านี้”

ข้อควรรู้: API ของกระทรวงพาณิชย์ (2 แหล่งแรก) **ตอบช้ามาก** บางครั้งใช้เวลา 30–45 วินาทีต่อครั้ง สคริปต์เลยมีระบบ “ลองใหม่” (retry) ถ้าครั้งแรกไม่สำเร็จ จะรออีกหน่อยแล้วลองอีก 2 ครั้ง

| 3. โครงสร้างโค้ด: แต่ละส่วนทำหน้าที่อะไร

โค้ดถูกแบ่งเป็นก้อน ๆ ให้อ่านง่าย แต่ละก้อนมีหน้าที่ชัดเจน ลองดูภาพรวมก่อน แล้วค่อยเจาะทีละส่วน:

ส่วนของโค้ด	หน้าที่ (พูดแบบบ้าน ๆ)
class Config	กล่องเก็บค่าตั้งต้น เช่น จะดึงปีไหน timeout กี่วินาที ปรับตรงนี้ทีเดียว
http_get_json()	“คนวิ่งของ” โทไปขอข้อมูลจาก API ถ้าพลาดก็ลองใหม่ให้
class ConstructionDataFetcher	“หัวหน้างาน” คุมการดึงทั้งหมด + เขียน Excel
.fetch_csi_national()	ดึงแหล่งที่ 1 (ดัชนีประเทศ)
	ดึงแหล่งที่ 2 (รายจังหวัด)
.fetch_cpip_provincial()	
.fetch_green_label()	ดึงแหล่งที่ 3 (ราคาจริง)
.write_excel()	รวมทุกอย่างเขียนลง Excel + จัดสวย
main()	“ปุ่มสตาร์ท” สั่งให้ทุกอย่างทำงานตามลำดับ

เราจะไล่ดูทีละส่วนในหน้าถัด ๆ ไป พร้อมโค้ดจริงและคำอธิบาย

| 4. เจาะส่วนที่ 1: Config — กล่องตั้งค่า

ส่วนนี้คือ “แผงควบคุม” ของสคริปต์ ถ้าอยากเปลี่ยนอะไร เปลี่ยนตรงนี้ที่เดียว ไม่ต้องไปไล่แก้ทั้งโค้ด

1	<code>class Config:</code>	
2	<code>OUTPUT_DIR = "output"</code>	<code># โฟลเดอร์เก็บไฟล์ Excel</code>
3	<code>TIMEOUT = 45</code>	<code># รอ API นานสุดกี่วินาที</code>
4	<code>RETRIES = 2</code>	<code># ลองใหม่อีกกี่ครั้งถ้าพลาด</code>
5	<code>RETRY_DELAY = 3</code>	<code># รอกี่วินาทีก่อนลองใหม่</code>
6	<code>PACE_DELAY = 1.0</code>	<code># หน่วง</code>
	<code>ระหว่าง request กัน rate-limit</code>	
7		
8	<code>CSI_FROM_YEAR = 2023</code>	<code># ดึง CSI ตั้งแต่ปีไหน</code>
9	<code>CSI_TO_YEAR = 2025</code>	<code># ถึงปีไหน</code>
10	<code>CPIP_FROM_YEAR = 2024</code>	
11	<code>CPIP_TO_YEAR = 2025</code>	

Listing 1: คลาส Config — ค่าตั้งต้นทั้งหมด

อ่านง่าย ๆ

อยากได้ข้อมูลย้อนหลังถึงปี 2020? เปลี่ยน `CSI_FROM_YEAR = 2020` จบ
API ตอบช้าจนพลาดบ่อย? เพิ่ม `TIMEOUT = 60` หรือ `RETRIES = 3`
ไม่ต้องเข้าใจโค้ดทั้งหมด แค่ว่าแต่ละค่าหมายถึงอะไรก็ปรับได้

I 5. เจาะส่วนที่ 2: http_get_json — คนวิ่งของ

ฟังก์ชันนี้มีหน้าที่เดียว: “ไปขอข้อมูลจาก URL ที่กำหนด แล้วเอากลับมา” แต่เพราะ API ราชการชอบตอบช้า/พลาด มันเลยมีระบบ **ลองใหม่อัตโนมัติ**

```

1 def http_get_json(url, timeout, retries, retry_delay, logger=
  print):
2     last_err = None
3     for attempt in range(retries + 1):           # ลอง
        ทั้งหมด retries+1 ครั้ง
4         try:
5             req = urllib.request.Request(url, headers={...})
6             with urllib.request.urlopen(req, timeout=timeout)
              as resp:
7                 raw = resp.read().decode("utf-8")
8                 return json.loads(raw)           # สำเร็จ! คืนข้อมูลเลย
9         except (URLError, TimeoutError, JSONDecodeError) as
              err:
10            last_err = err
11            if attempt < retries:
12                logger(f"retry {attempt+1}/{retries} ...")
13                time.sleep(retry_delay)           # รอแล้วลองใหม่
14            return None                           # ลองครบแล้วยัง
        พลาด -> คืน None

```

Listing 2: http_get_json — ขอข้อมูลพร้อม retry

เล่าเป็นเรื่อง: เหมือนเราโทรสั่งพิซซ่า ถ้าสายไม่ว่า เราไม่ได้เลิกสั่ง เราวางสาย รอแป็บ แล้วโทรใหม่ ลองสัก 3 ครั้ง ถ้ายังไม่ติดจริง ๆ ค่อยยอมแพ้ (คืนค่า None แปลว่า “ไม่ได้ของ”)

ทำไมต้องคืน None ไม่ใช่ crash? เพราะถ้าแหล่งหนึ่งล่ม เรายังอยากได้ข้อมูลจากอีก 2 แหล่งที่เหลือ การคืน None ทำให้สคริปต์ “ข้าม” แหล่งที่พังไปได้ ไม่ทำให้ทั้งโปรแกรมตาย

| 6. เจาะส่วนที่ 3: การดึงแต่ละแหล่ง

หัวหน้างาน (ConstructionDataFetcher) มีลูกน้อง 3 คน แต่ละคนดึงคนละแหล่ง หลักการเหมือนกันหมด: สร้าง URL → เรียก http_get_json → แปลงเป็นตาราง (DataFrame)

```

1  def fetch_csi_national(self):
2      url = (f"{self.MOC_BASE}/csi-indexes"
3            f"?index_id=0000000000000000"      # 0000... = ดัชนี
4            รวม
5            f"&from_year={Config.CSI_FROM_YEAR}"
6            f"&to_year={Config.CSI_TO_YEAR}")

7      data = http_get_json(url, ...)          # เรียกคนวิ่งของ
8      if not data:
9          return None                        # ไม่ได้ก็ข้าม
10
11     df = pd.DataFrame(data)                 # แปลง JSON ->
12     ตาราง
13     df = self._add_thai_date(df)            # เพิ่ม
14     คอลัมน์ period
15     return df

```

Listing 3: fetch_csi_national — ดึงดัชนีระดับประเทศ

Green Label พิเศษหน่อย เพราะข้อมูลที่ได้ซ่อนอยู่ลึกกว่า ต้องเจาะเข้าไปที่ result → items และราคายังเป็นข้อความ “2,731.10” (มีลูกน้ำ) ต้องแปลงเป็นตัวเลขก่อนคำนวณได้:

```

1  # "2,731.10" (str) -> 2731.10 (float)
2  df["price_baht_numeric"] = (
3      df["price_baht"].astype(str)
4      .str.replace(",", "", "")          # เอาลูกน้ำออก
5      .apply(lambda x: pd.to_numeric(x, errors="coerce"))
6  )

```

Listing 4: แปลงราคาข้อความเป็นตัวเลข

| 7. เจาะส่วนที่ 4: write_excel — เขียนไฟล์ + จัดสว

พอดึงข้อมูลครบแล้ว ส่วนนี้รวมทุกอย่างเขียนลง Excel ไฟล์เดียว แยกเป็นหลาย sheet แล้วจัด format ให้สวย (หัวตารางสีน้ำเงิน ตัวอักษรขาว ปรับความกว้างคอลัมน์อัตโนมัติ)

```

1 def write_excel(self, frames):
2     ts = self.run_started.strftime("%Y%m%d_%H%M%S")
3     filename = f"construction_data_{ts}.xlsx" # ชื่อ
        มี timestamp กันชนกัน
4
5     with pd.ExcelWriter(filepath, engine="openpyxl") as writer:
6         self._write_summary_sheet(writer, frames) # sheet
            สรุ
7         # เขียนแต่ละแหล่งเป็น sheet แยก
8         csi.to_excel(writer, sheet_name="CSI_National")
9         cpip.to_excel(writer, sheet_name="CPIP_Provincial")
10        green.to_excel(writer, sheet_name="GreenLabel_Prices")
11        # เขียน log การดึงทั้งหมด
12        logs_df.to_excel(writer, sheet_name="Fetch_Logs")
13
14        self._style_workbook(filepath) # จัดสวที่หลัง

```

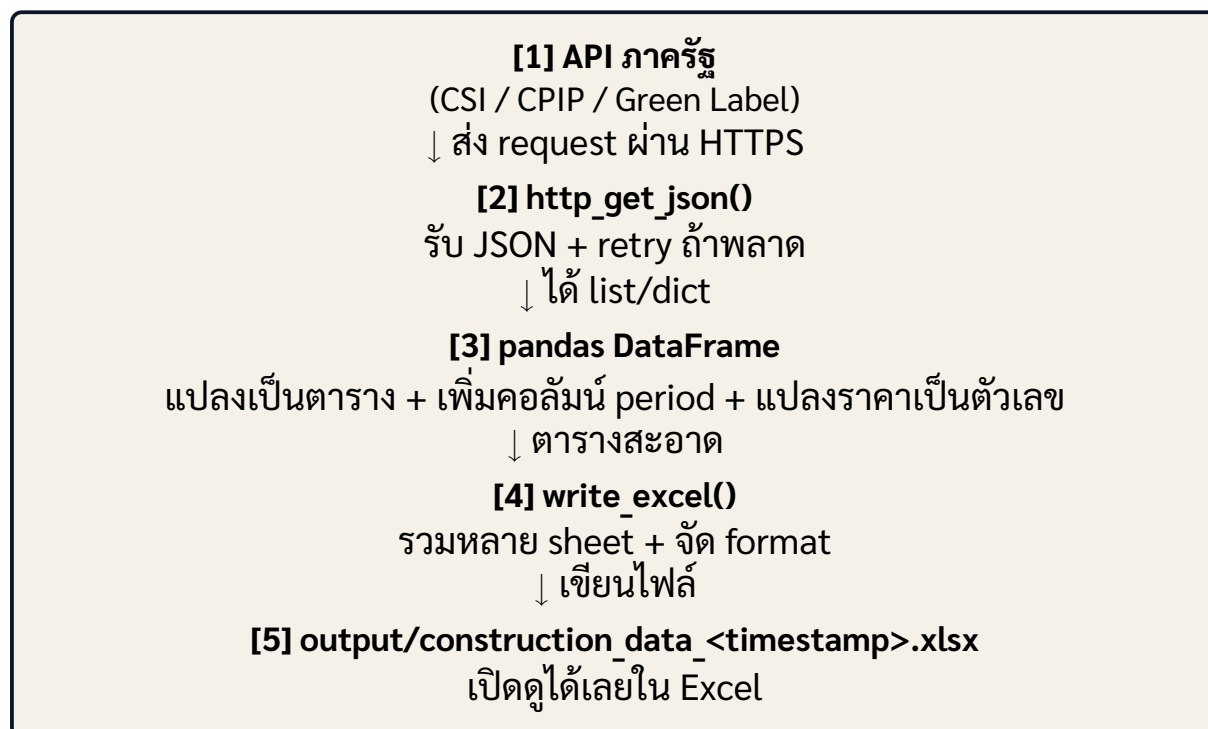
Listing 5: write_excel — เขียนหลาย sheet

ทำไมชื่อไฟล์มี timestamp?

เพราะเราอยากเก็บประวัติทุกครั้งที่ตั้ง ไม่ให้ทับกัน ถ้ารันวันนี้ได้ construction_data_20260610_154817.xlsx รันพรุ่งนี้ก็ได้ชื่อใหม่ ทำให้ย้อนดูได้ว่าข้อมูลเดือนนี้ต่างจากเดือนก่อนยังไง

| 8. Data Pipeline: เส้นทางข้อมูลตั้งแต่ต้นจนจบ

ภาพรวมว่าข้อมูล “ไหล” จากต้นทางถึงปลายทางอย่างไร:



สรุปเป็นประโยคเดียว: **API → ดึง+ลงใหม่ → จัดตาราง → เขียน Excel → เปิดดู**

| 9. ตัวอย่างจริง: Input / Process / Output

มาดูตัวอย่างจริง ตั้งแต่สิ่งที่เราใส่เข้าไป จนถึงสิ่งที่ได้ออกมา

9.1 INPUT — สิ่งที่เราสั่ง

```
$ python3 fetch_construction_data.py
```

Listing 6: แคร์รันคำสั่งเดียว

ไม่ต้องใส่ argument อะไรเลย ค่าทั้งหมดอยู่ใน Config แล้ว (อยากเปลี่ยนปีก็ไปแก้ใน Config ก่อนรัน)

9.2 PROCESS — สิ่งที่เกิดขึ้นระหว่างรัน

สคริปต์จะพ่น log ออกมาให้เห็นว่ากำลังทำอะไร (ตัวอย่างจริงจากการรัน):

```
[1/3] CSI - ดัชนีราคาวัสดุก่อสร้างระดับประเทศรายเดือน ()
      GET https://dataapi.moc.go.th/csi-indexes?index_id=00...
      retry 1/2 (TimeoutError) ...
      [OK] 24 แถว (2023/1 - 2024/12)
[2/3] CPIP - ดัชนีราคาก่อสร้างรายจังหวัด (77 จังหวัด)
      [OK] 728 แถว, 61 จังหวัด
[3/3] Green Label - ราคาวัสดุลากเขียวจริงบาท ()
      [OK] 5571 รายการ, 12 หมวดหมู่
```

9.3 OUTPUT — สิ่งที่ได้ (ข้อมูลจริง)

ตัวอย่างข้อมูล CSI ที่ได้ (ดัชนีราคาวัสดุก่อสร้างระดับประเทศ):

period	price_index	yoy (%)	ความหมาย
2023-01	113.1	+3.5	แพงขึ้น 3.5% จากปีก่อน
2023-12	112.0	-0.4	ถูกลงเล็กน้อย
2024-12	112.4	+0.4	ทรงตัว

ตัวอย่างข้อมูล CPIP ที่ได้ (ดัชนีรายจังหวัด, ม.ค. 2024):

จังหวัด	period	price_index
พังงา	2024-01	106.7
นนทบุรี	2024-01	109.1
เชียงราย	2024-01	107.3

ตัวอย่างข้อมูล Green Label ที่ได้ (ราคาจริงเป็นบาท):

ยี่ห้อ	วัสดุ	หน่วย	ราคา (บาท)
ซีแพค	วัสดุก่อ (คอนกรีต 400)	ลบ.ม.	2,731.10
ซีแพค	วัสดุก่อ (คอนกรีต 380)	ลบ.ม.	2,667.40

ไฟล์ Excel ที่ได้ มี 6 sheet:

ชื่อ sheet	เนื้อหา
Summary	สรุปภาพรวมการดึง (เวลา, จำนวนแถวแต่ละแหล่ง)
CSI_National	ดัชนีระดับประเทศ 24 แถว
CPIP_Provincial	ดัชนีรายจังหวัด 728 แถว
GreenLabel_Prices	ราคาวัสดุจริง 5,571 รายการ
GreenLabel_Summary	สรุปราคา เฉลี่ย/ต่ำสุด/สูงสุด ต่อหมวด
Fetch_Logs	log การดึงทุกขั้นตอน

| 10. วิธีใช้งานจริง

ขั้นที่ 1 — ติดตั้ง library ที่ต้องใช้

```
pip install -r requirements.txt  
# หรือ  
pip install pandas openpyxl
```

ขั้นที่ 2 — รันสคริปต์

```
cd data_fetcher  
python3 fetch_construction_data.py
```

ขั้นที่ 3 — เปิดไฟล์

ไฟล์จะอยู่ในโฟลเดอร์ output/ ซึ่งมี timestamp เปิดด้วย Excel / Numbers / Google Sheets ได้เลย

สรุปสั้น ๆ

สคริปต์นี้เปลี่ยนงานที่เคยต้องนั่งทำมือหลายชั่วโมง (เข้าเว็บ โหลด PDF อ่าน จดใส่ Excel) ให้เหลือแค่ **รันคำสั่งเดียว** แล้วได้ไฟล์ Excel พร้อมข้อมูลราคาวัสดุจริงจากภาครัฐ 3 แหล่ง รวมกว่า 6,300 แถว ภายในไม่กี่นาที — และทำซ้ำได้ทุกเมื่อที่อยากได้ข้อมูลใหม่